

Computer Vision and Machine Learning for Scientific Visual Data Analytics

Lecture 3: Classification



Shu Kong
shu.kong@oist.jp

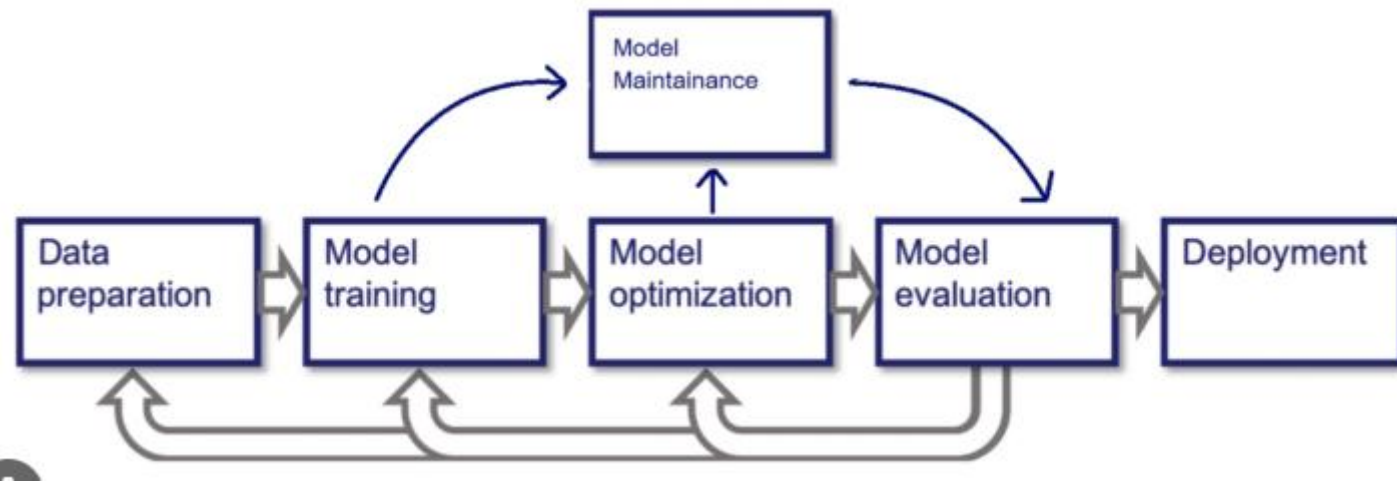


Roadmap

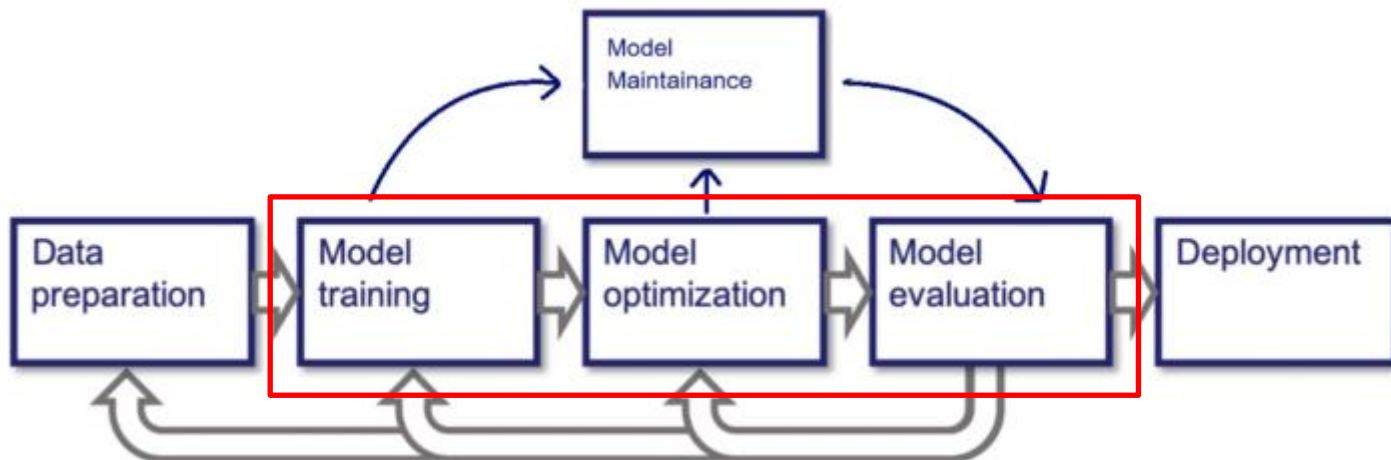
Teaching materials are available: <https://github.com/aimerykong/OIST-mini-course-CVML>

Lecture	Topics	Hands-on experience
1. Intro	CV/ML, deep learning, artificial intelligence	Configure programming environment using Google Drive and Colab; implement basic scripts in Python
2. Basics of CV/ML	training, validation, testing, features, metrics, nearest neighbor	Implement a machine learning pipeline for image classification
 3. Classification	Logistic Regression, gradient descent	Train a linear classifier for pollen species recognition
4. Deep Learning	Multi-Layer Perceptron, feature learning, backpropagation, neural networks	Train a convolutional neural network (CNN) for pollen species recognition
5. Segmentation and Detection	semantic and instance segmentation, U-Net,	Train a neural network for pollen grain detection/segmentation
6. Foundation Models	CLIP , Segment Anything Model , DINOv2 , GroundingDINO , finetuning	Finetune a pretrained model for image classification and segmentation
7. Advanced Topics	open-set recognition, novel species recognition, phylogentic reconstruction via clustering	Implement algorithms to recognize unknown/anomalous/novel observations
8. Brainstorm & Talks	Volunteer to present	n/a

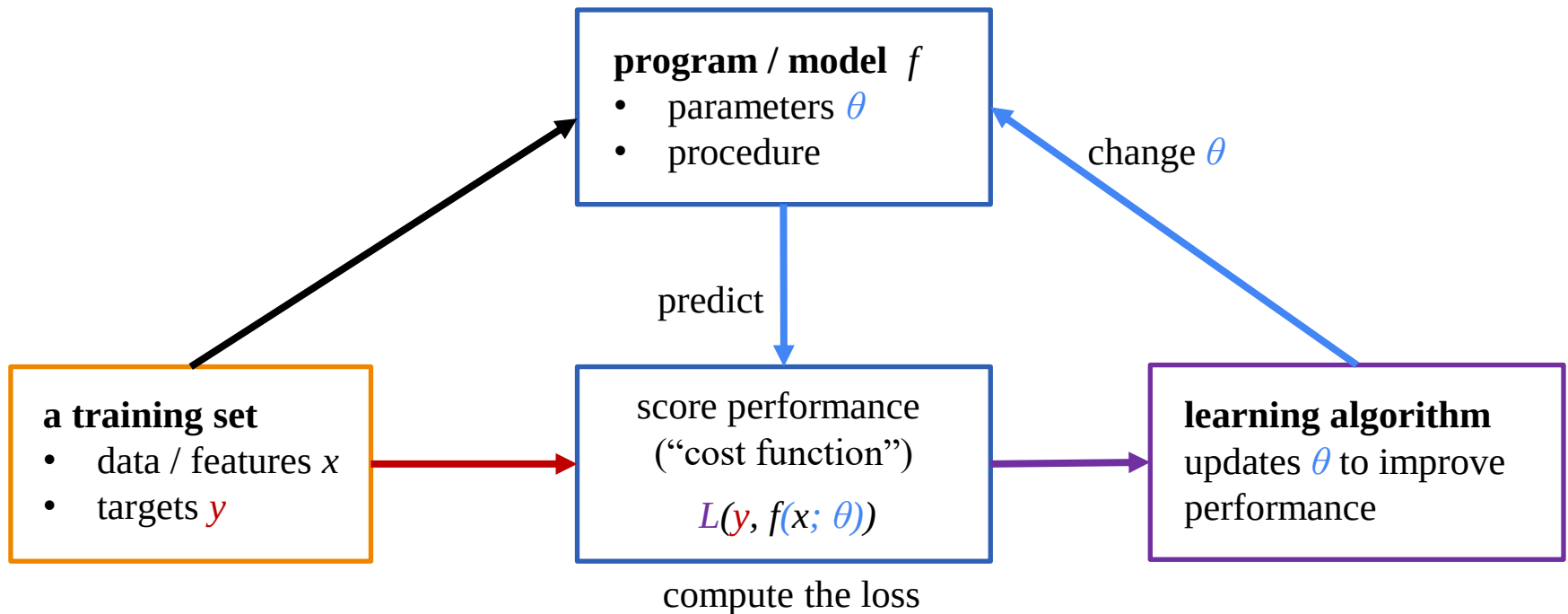
Recap: An Overall Pipeline of Machine Learning



Recap: An Overall Pipeline of Machine Learning



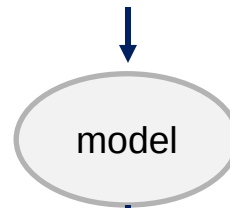
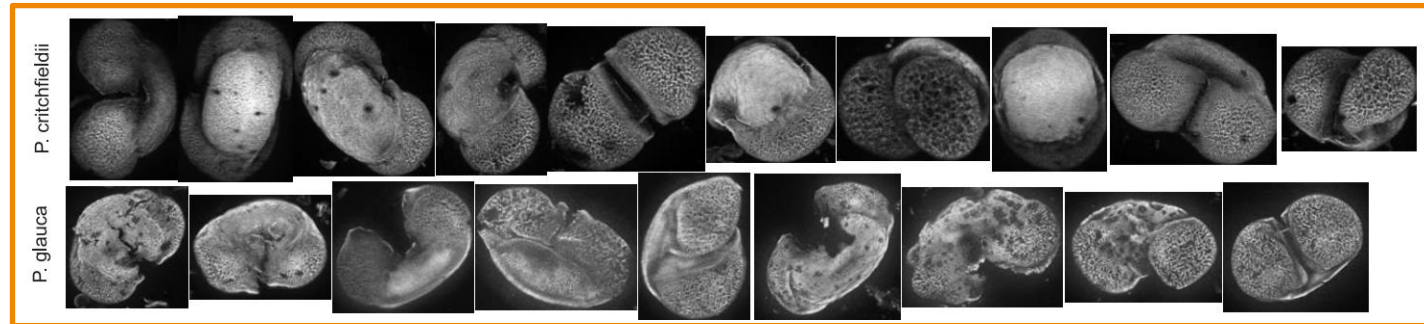
Recap: An Overall Pipeline of Machine Learning



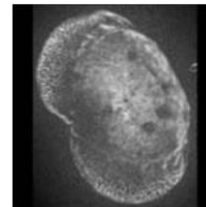
Recap: labeled data

a training set

- data *features* x
- targets $y \in \{1, 2, \dots\}$



Train a model (i.e., teach a machine) to recognize the species of a test image x'



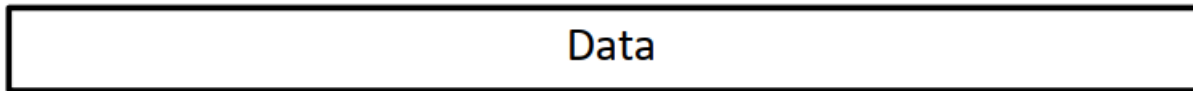
x'
class-1 or class-2?

feature x :

size of (a)
shape of (a)
size of (b)
shape of (b)
size of (c)
shape of (c)
texture-1
texture-2
.....

Raw pixels as features? Sure!
Enhanced pixels as features? Fine!
Manually designed features? Good!
Auto-learned features? Great!

Recap: labeled data and features



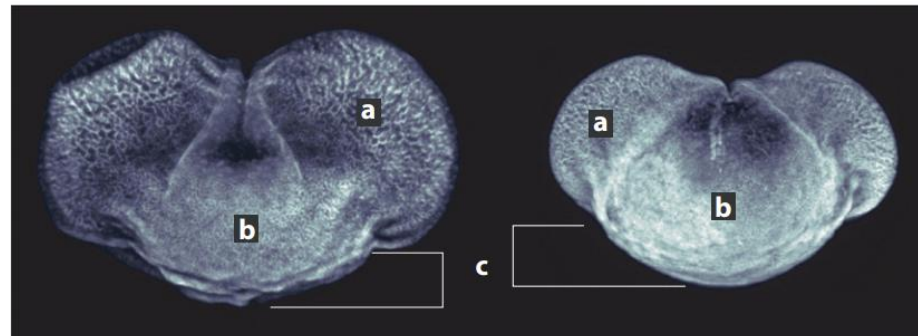
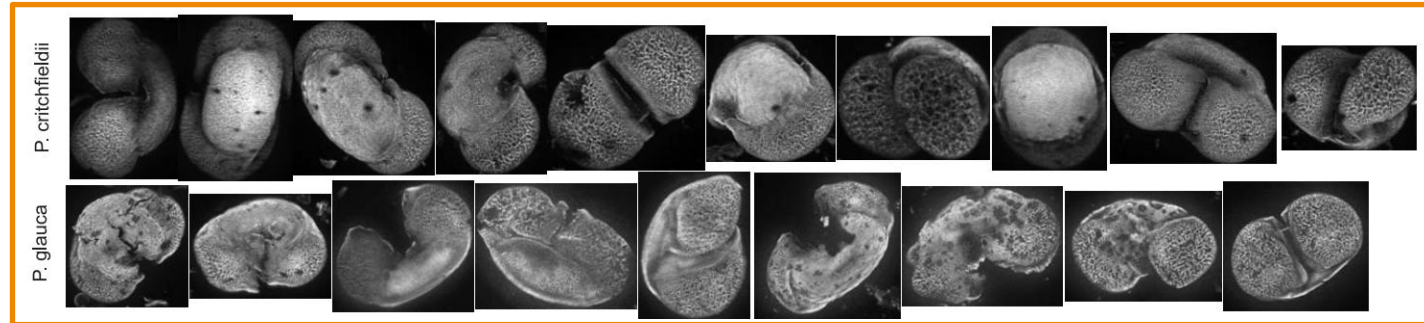
- Several candidate learning algorithms or models, each of which can be fit to data and used for prediction
- How can we decide which is best?



Recap: labeled data and features

a training set

- data *features* x
- targets $y \in \{1, 2, \dots\}$



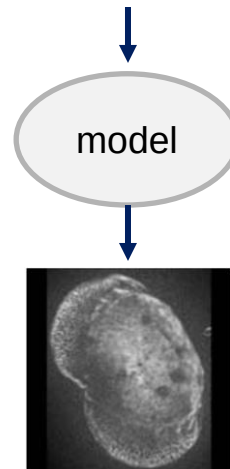
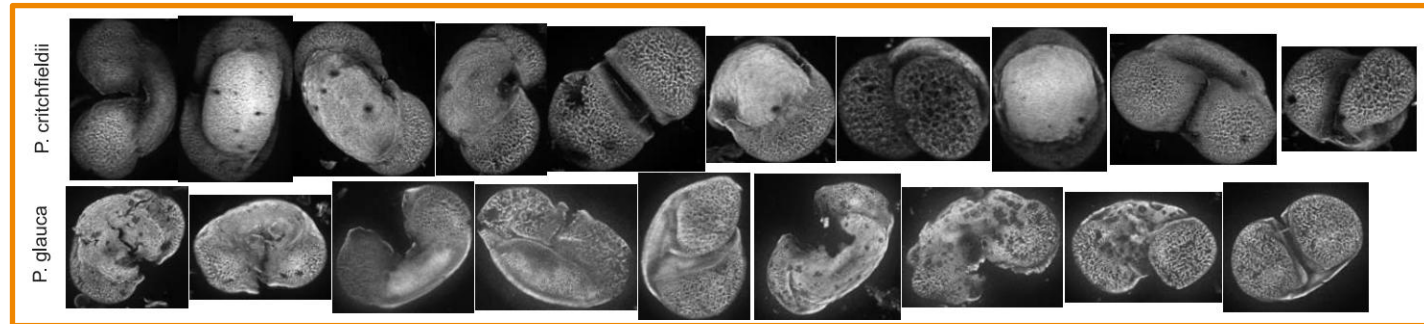
feature x :

size of (a)
shape of (a)
size of (b)
shape of (b)
size of (c)
shape of (c)
texture-1
texture-2
.....

Recap: labeled data and features

a training set

- data *features* x
- targets $y \in \{1, 2, \dots\}$



Train a model (i.e., teach a machine) to recognize the species of a test image x'

x'
class-1 or class-2?

feature x :

size of (a)
shape of (a)
size of (b)
shape of (b)
size of (c)
shape of (c)
texture-1
texture-2
.....

Recap: Nearest Neighbor

Find the nearest neighbor to x' in the training dataset

a training set

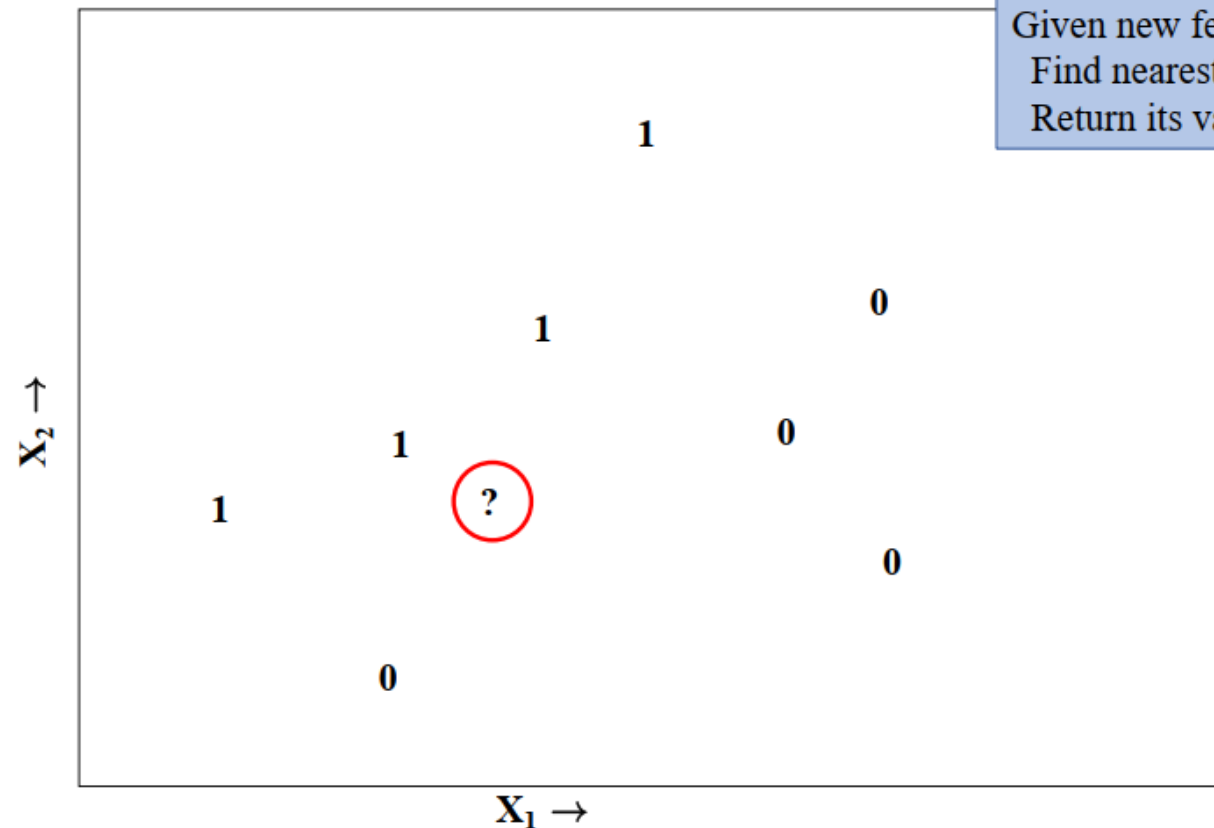
- data *features* x
- targets $y \in \{1, 2, \dots\}$

“Predictor”:

Given new features:
Find nearest example
Return its value

feature x :

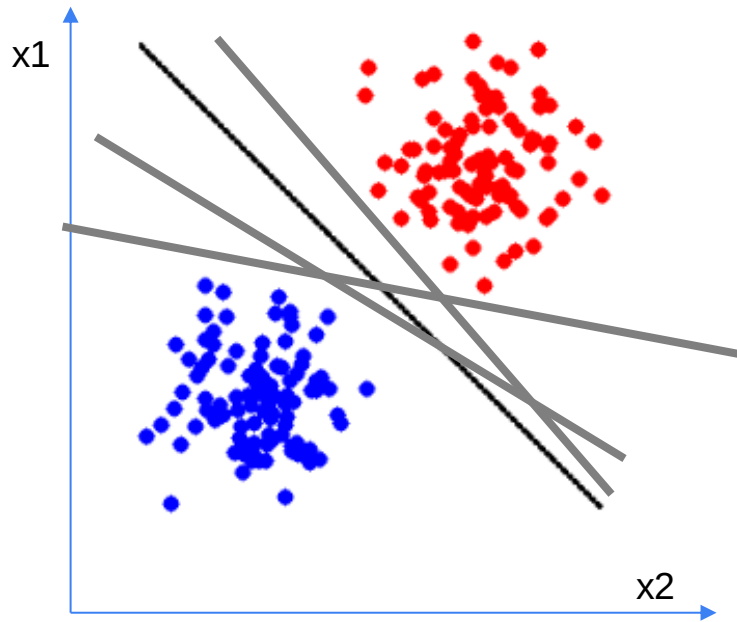
size of (a)
shape of (a)
size of (b)
shape of (b)
size of (c)
shape of (c)
texture-1
texture-2
.....



Outline

- **Linear Classification and Perceptron Learning**
- Logistic Regression and Gradient-Based Learning
- Multi-Class Classification

Linear Classifier



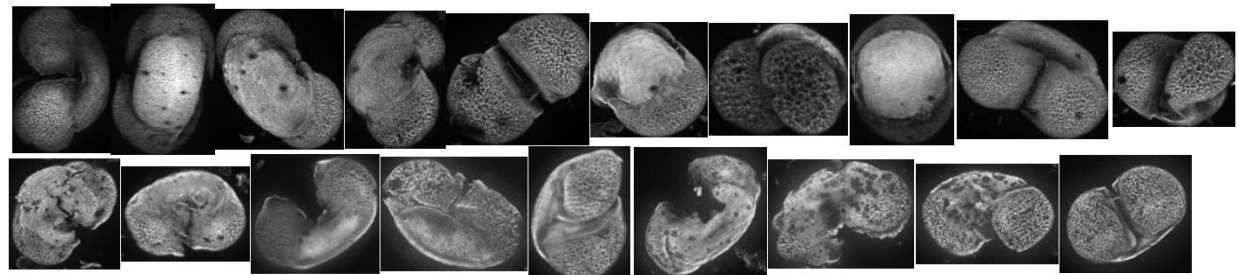
a training set

- data / features x
- targets y

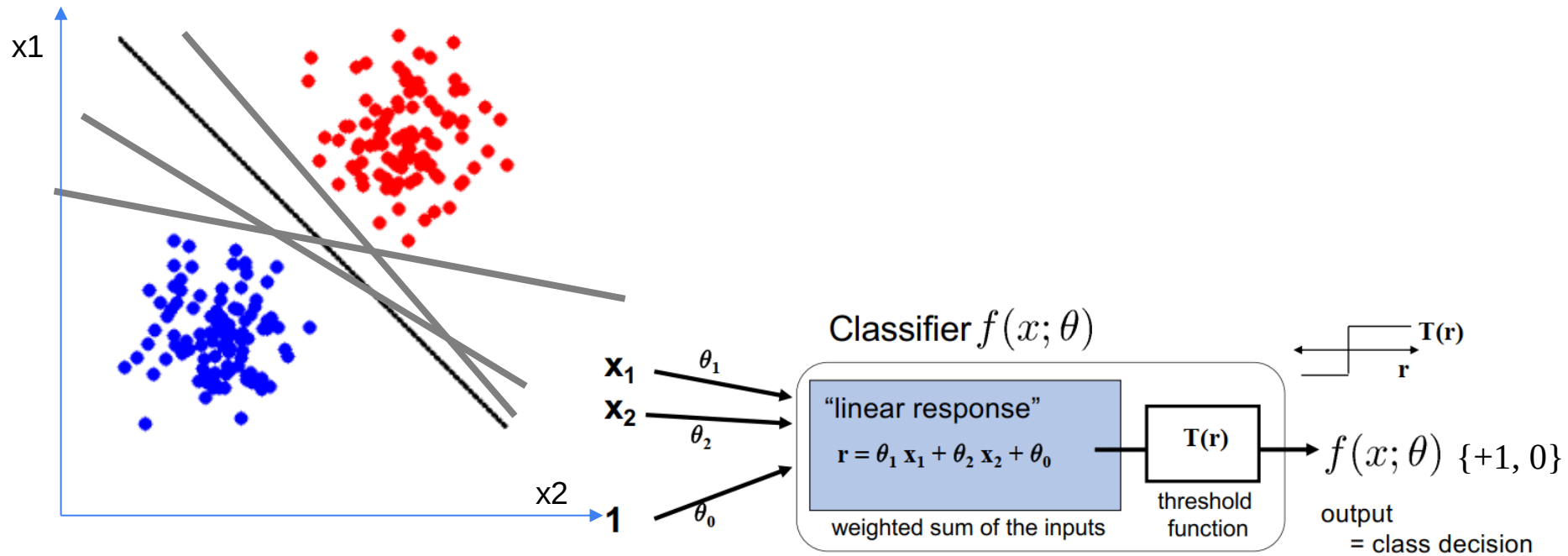
feature x :

size of (a)
shape of (a)
size of (b)
shape of (b)
size of (c)
shape of (c)
texture-1
texture-2
.....

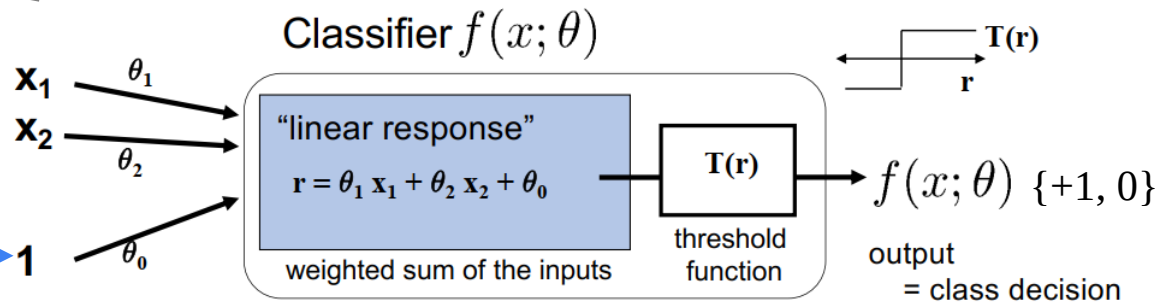
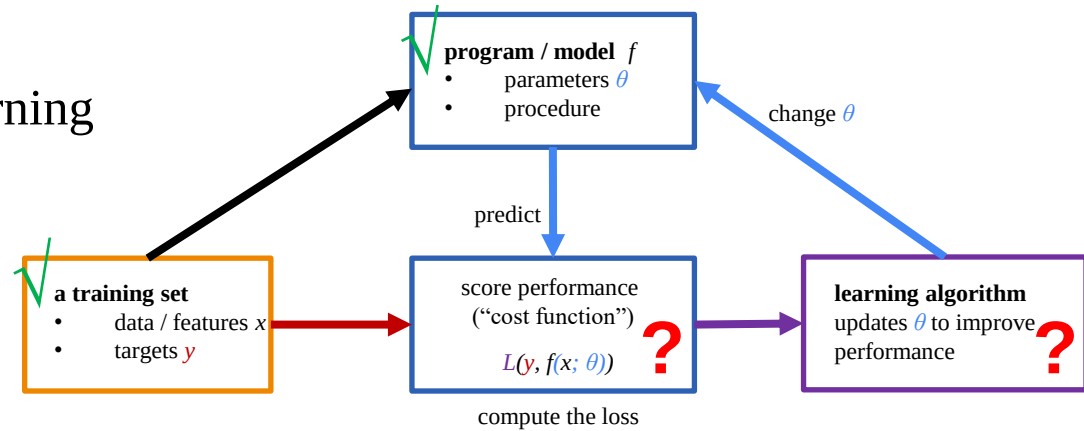
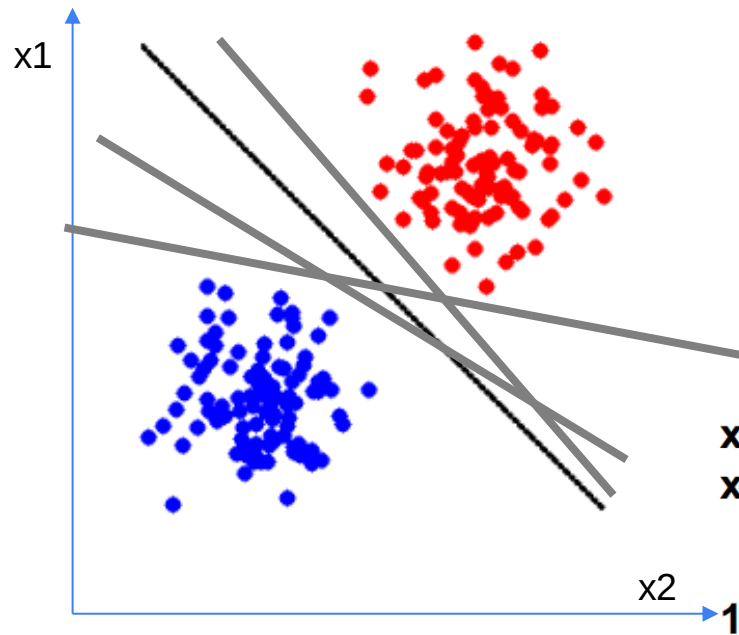
P. critchfieldii
P. glauca



Linear Classifier: Perceptron Learning



Linear Classifier: Perceptron Learning



- **Learning from Training Data:**
 - training data = labeled feature vectors
 - Find parameter values that predict well (low error)
 - error is estimated on the training data
 - “true” error will be on future test data
- **Define a loss function $J(\theta)$:**
 - Classifier error rate (for a given set of weights θ and labeled data)
- **Minimize this loss function (or, maximize accuracy)**
 - An optimization or search problem over the vector $(\theta_0, \theta_1, \theta_2, \dots)$

Linear Classifier: Perceptron Learning

- How should we measure error?
 - Natural measure = “fraction we get wrong” (error rate)

$$\text{err}(\theta) = \frac{1}{m} \sum_i \mathbb{1}[y^{(i)} \neq f(x^{(i)}; \theta)] \quad \text{where} \quad \mathbb{1}[y \neq \hat{y}] = \begin{cases} 1 & y \neq \hat{y} \\ 0 & \text{o.w.} \end{cases}$$

Linear Classifier: Perceptron Learning

- How should we measure error?
 - Natural measure = “fraction we get wrong” (error rate)

$$\text{err}(\theta) = \frac{1}{m} \sum_i \mathbb{1}[y^{(i)} \neq f(x^{(i)}; \theta)] \quad \text{where} \quad \mathbb{1}[y \neq \hat{y}] = \begin{cases} 1 & y \neq \hat{y} \\ 0 & \text{o.w.} \end{cases}$$

- Perceptron algorithm: an ~~SGD-like~~ algorithm

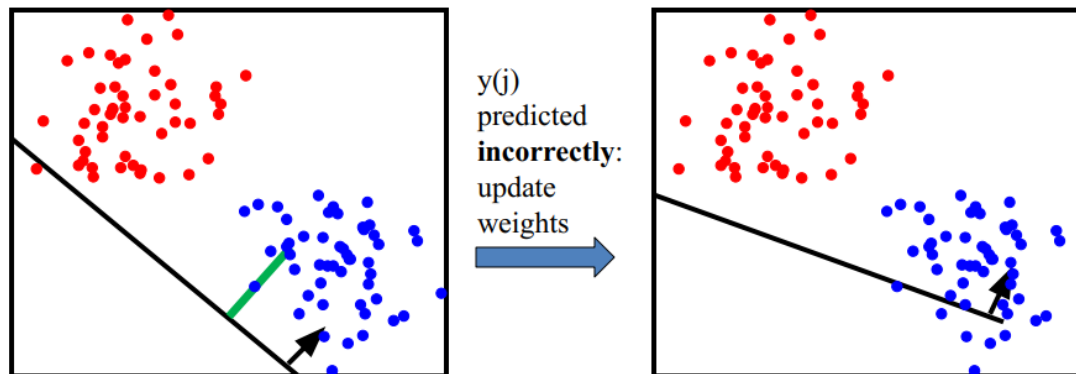
Freund, Y.; Schapire, R. E. (1999).

while $\neg$ done:

for each data point j :

$$\hat{y}^{(j)} = \text{sign}(\theta \cdot x^{(j)}) \quad \text{(predict output for point } j\text{)}$$

$$\theta \leftarrow \theta + \alpha(y^{(j)} - \hat{y}^{(j)})x^{(j)} \quad \text{("gradient-like" step)}$$



Linear Classifier: Perceptron Learning

- How should we measure error?
 - Natural measure = “fraction we get wrong” (error rate)

$$\text{err}(\theta) = \frac{1}{m} \sum_i \mathbb{1}[y^{(i)} \neq f(x^{(i)}; \theta)] \quad \text{where} \quad \mathbb{1}[y \neq \hat{y}] = \begin{cases} 1 & y \neq \hat{y} \\ 0 & \text{o.w.} \end{cases}$$

- Perceptron algorithm: an ~~SGD-like~~ algorithm

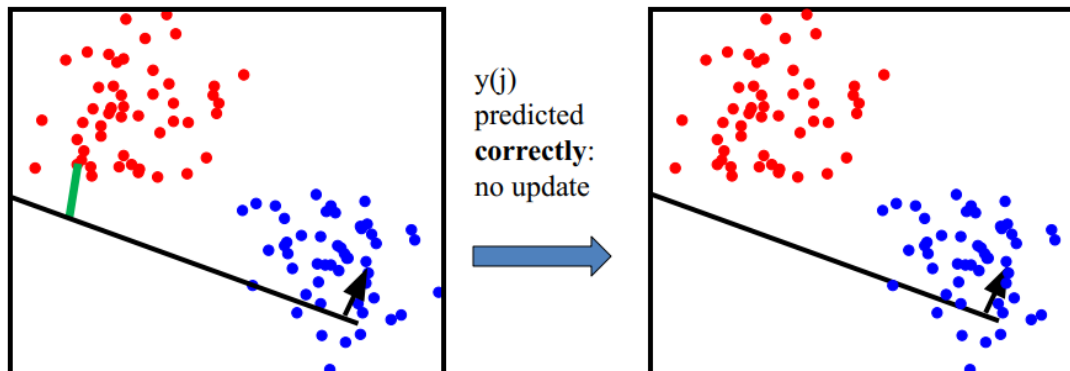
Freund, Y.; Schapire, R. E. (1999).

while $\neg$ done:

for each data point j :

$$\hat{y}^{(j)} = \text{sign}(\theta \cdot x^{(j)}) \quad (\text{predict output for point } j)$$

$$\theta \leftarrow \theta + \alpha(y^{(j)} - \hat{y}^{(j)})x^{(j)} \quad (\text{“gradient-like” step})$$



Linear Classifier: Perceptron Learning

- How should we measure error?
 - Natural measure = “fraction we get wrong” (error rate)

$$\text{err}(\theta) = \frac{1}{m} \sum_i \mathbb{1}[y^{(i)} \neq f(x^{(i)}; \theta)] \quad \text{where} \quad \mathbb{1}[y \neq \hat{y}] = \begin{cases} 1 & y \neq \hat{y} \\ 0 & \text{o.w.} \end{cases}$$

- Perceptron algorithm: an ~~SGD-like~~ algorithm

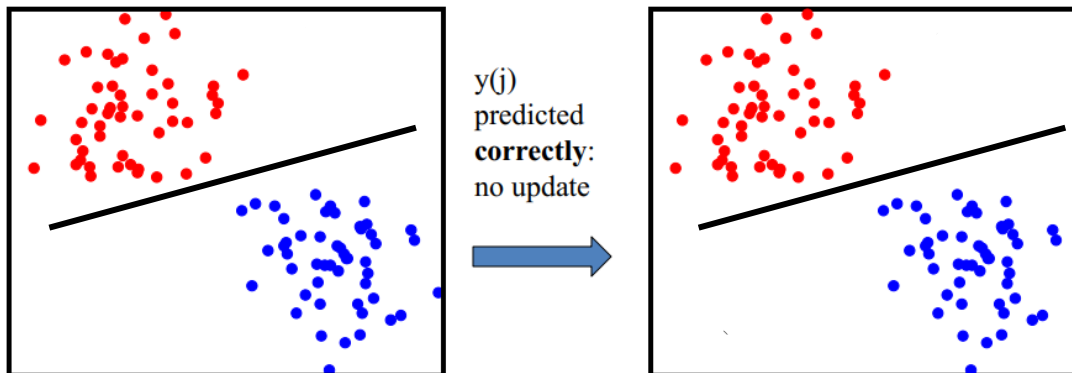
Freund, Y.; Schapire, R. E. (1999).

while $\neg$ done:

for each data point j :

$$\hat{y}^{(j)} = \text{sign}(\theta \cdot x^{(j)}) \quad \text{(predict output for point } j\text{)}$$

$$\theta \leftarrow \theta + \alpha(y^{(j)} - \hat{y}^{(j)})x^{(j)} \quad \text{("gradient-like" step)}$$



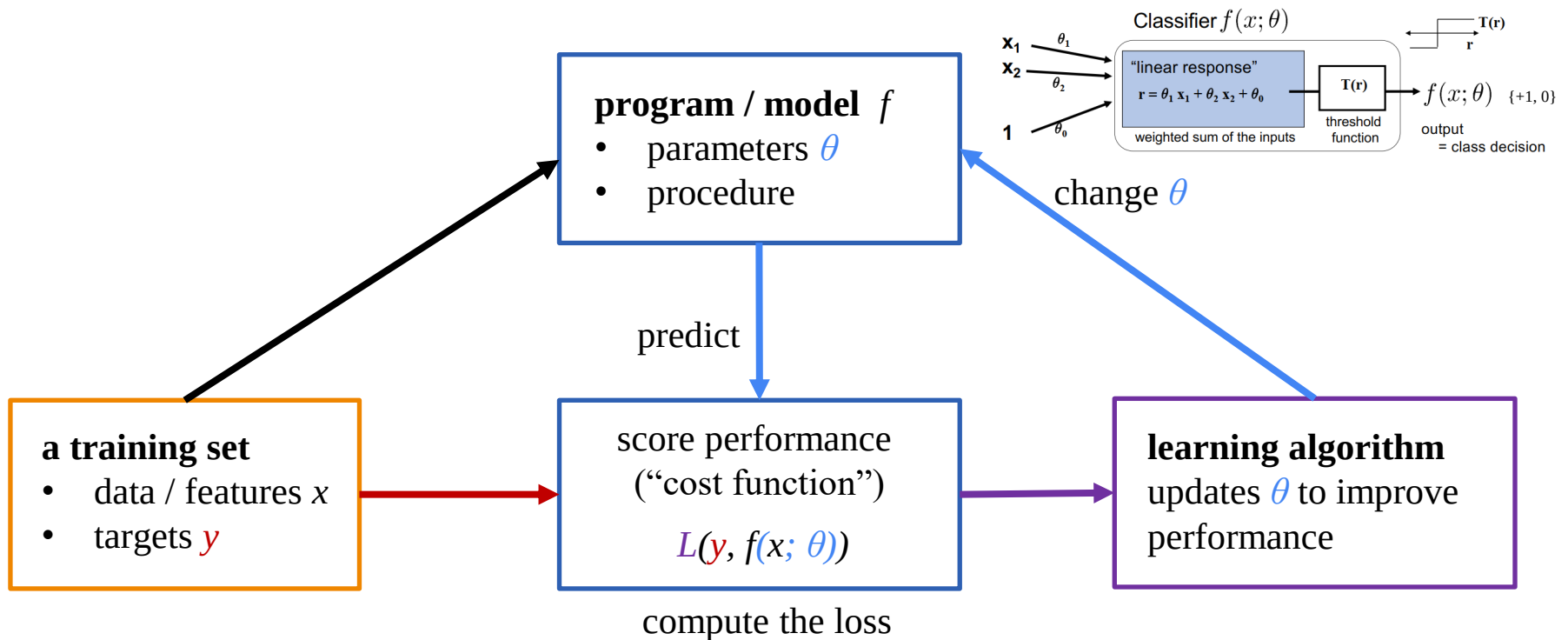
Perceptron Learning Algorithm in the Machine Learning pipeline

while $\neg$ done:

for each data point j :

$$\hat{y}^{(j)} = \text{sign}(\theta \cdot x^{(j)}) \quad \text{(predict output for point j)}$$

$$\theta \leftarrow \theta + \alpha(y^{(j)} - \hat{y}^{(j)})x^{(j)} \quad \text{("gradient-like" step)}$$



$$\text{err}(\theta) = \frac{1}{m} \sum_i \mathbb{1}[y^{(i)} \neq f(x^{(i)}; \theta)]$$

Perceptron Learning Algorithm

while $\neg$ done:

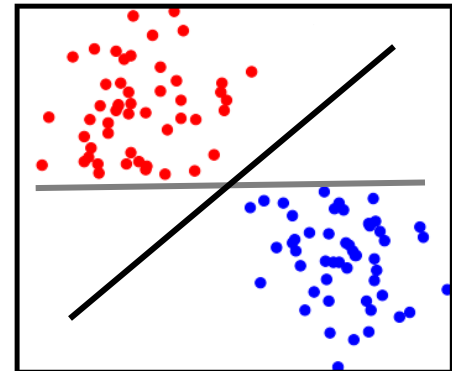
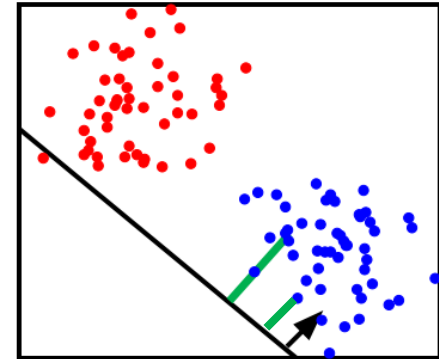
 for each data point j :

$$\hat{y}^{(j)} = \text{sign}(\theta \cdot x^{(j)}) \quad \text{(predict output for point } j \text{)}$$

$$\theta \leftarrow \theta + \alpha(y^{(j)} - \hat{y}^{(j)})x^{(j)} \quad \text{("gradient-like" step)}$$

Issues

- It does not consider “more right / more wrong”
- It does not distinguish good vs. better classifiers

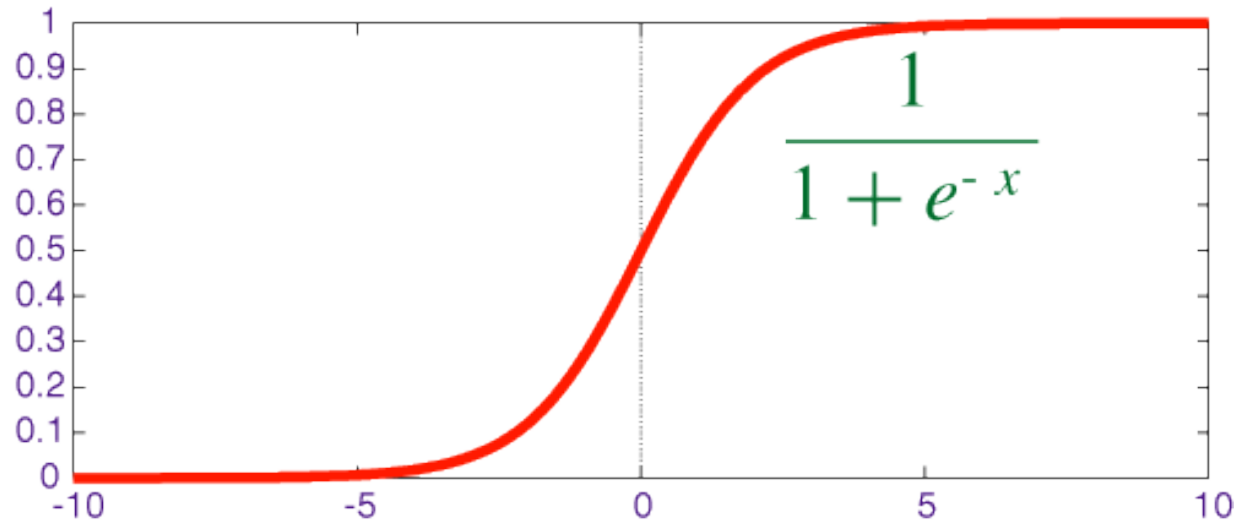
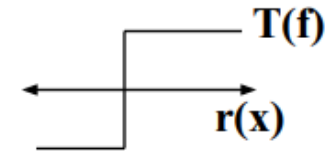


Outline

- Linear Classification and Perceptron Learning
- **Logistic Regression and Gradient-Based Learning**
- Multi-Class Classification

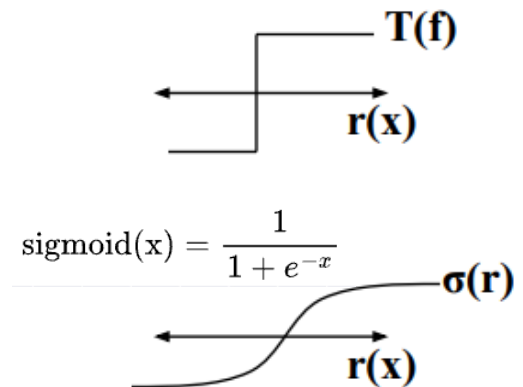
Surrogate Loss Functions

- Another solution: use a “smooth” loss
 - e.g., approximate the threshold function
 - Usually some smooth function of distance
 - Example: logistic “sigmoid”, looks like an “S”



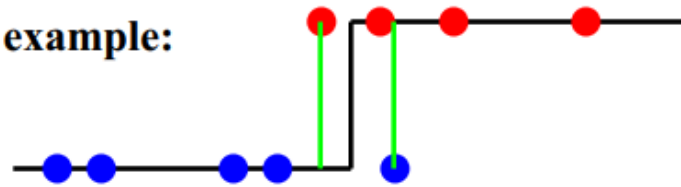
Surrogate Loss Functions

- Another solution: use a “smooth” loss
 - e.g., approximate the threshold function
 - Usually some smooth function of distance
 - Example: logistic “sigmoid”, looks like an “S”
 - Now, measure e.g. Mean Square Error (MSE)

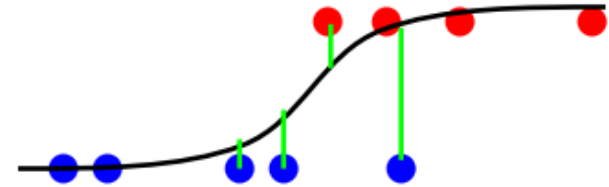


$$J(\underline{\theta}) = \frac{1}{m} \sum_j \left(\sigma(r(x^{(j)})) - y^{(j)} \right)^2$$

1D example:



Classification error = 2/9



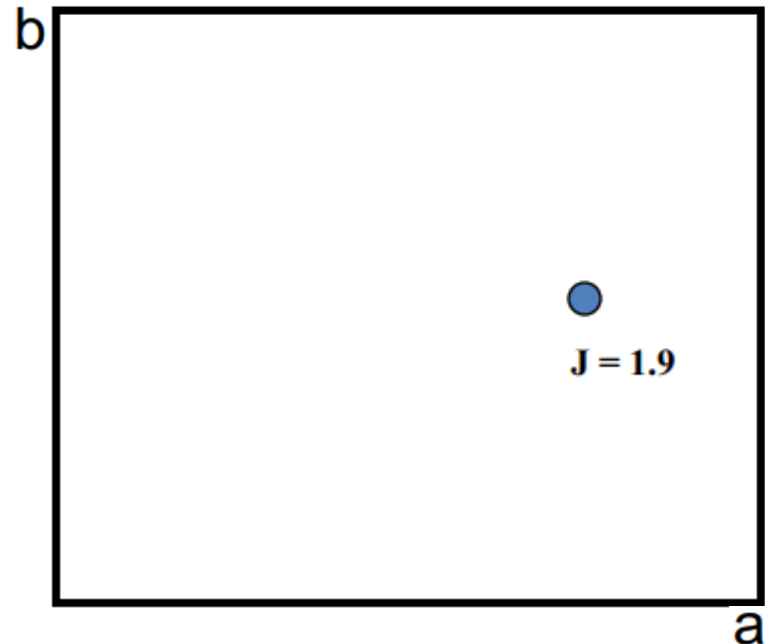
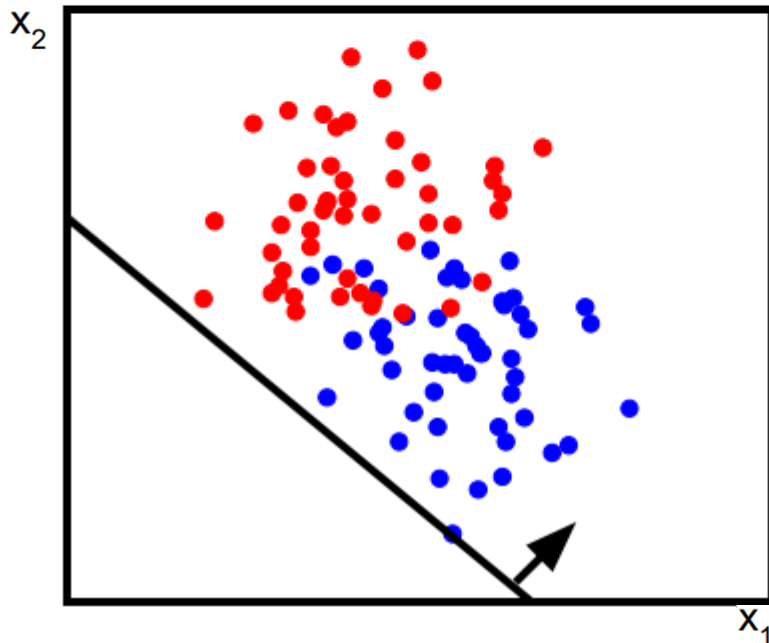
MSE = $(0^2 + 1^2 + .2^2 + .25^2 + .05^2 + \dots)/9$

Training the Classifier

- Once we have a smooth measure of quality, we can find the “best” settings for the parameters of

$$r(x_1, x_2) = a \cdot x_1 + b \cdot x_2 + c \quad J(\underline{\theta}) = \frac{1}{m} \sum_j \left(\sigma(r(x^{(j)})) - y^{(j)} \right)^2$$

- Example: 2D feature space $\Leftrightarrow$ parameter space

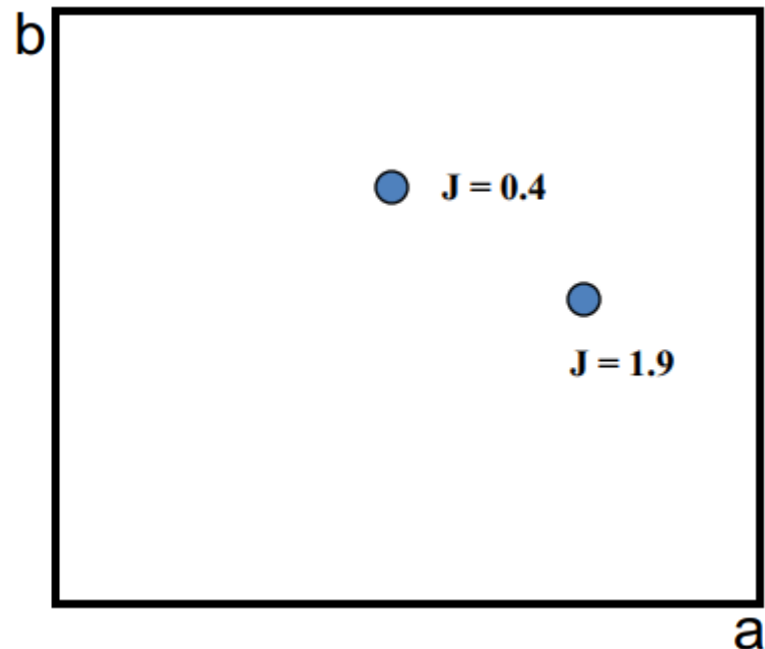
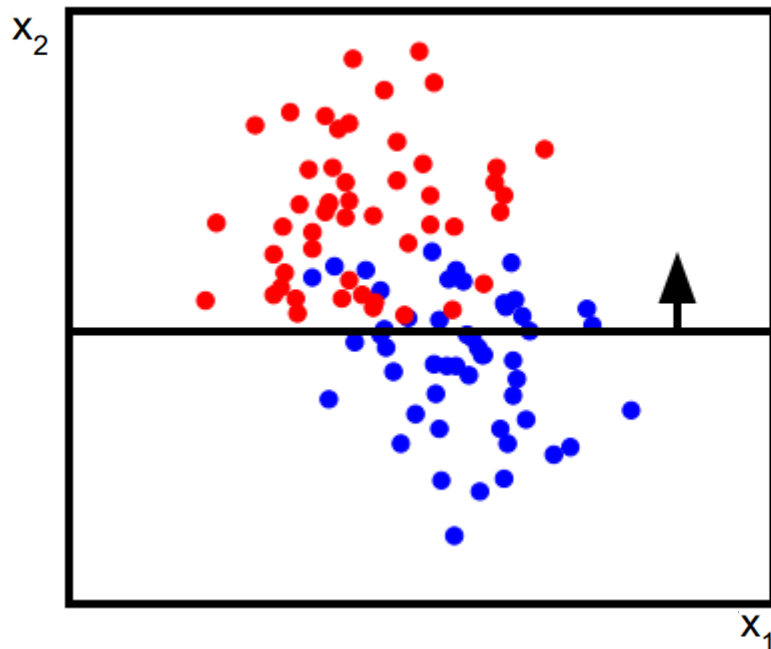


Training the Classifier

- Once we have a smooth measure of quality, we can find the “best” settings for the parameters of

$$r(x_1, x_2) = a \cdot x_1 + b \cdot x_2 + c \quad J(\underline{\theta}) = \frac{1}{m} \sum_j \left(\sigma(r(x^{(j)})) - y^{(j)} \right)^2$$

- Example: 2D feature space $\Leftrightarrow$ parameter space

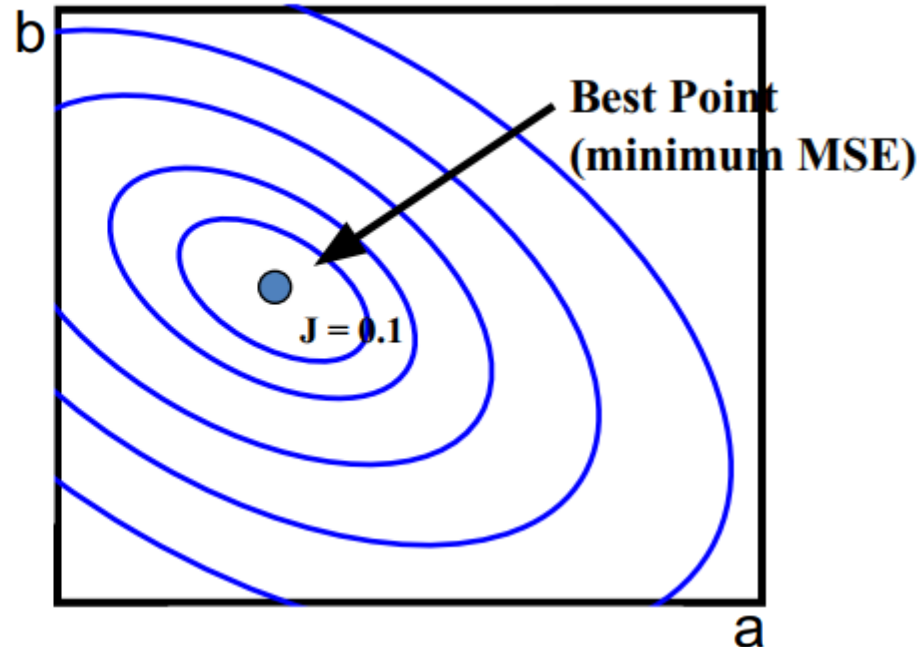
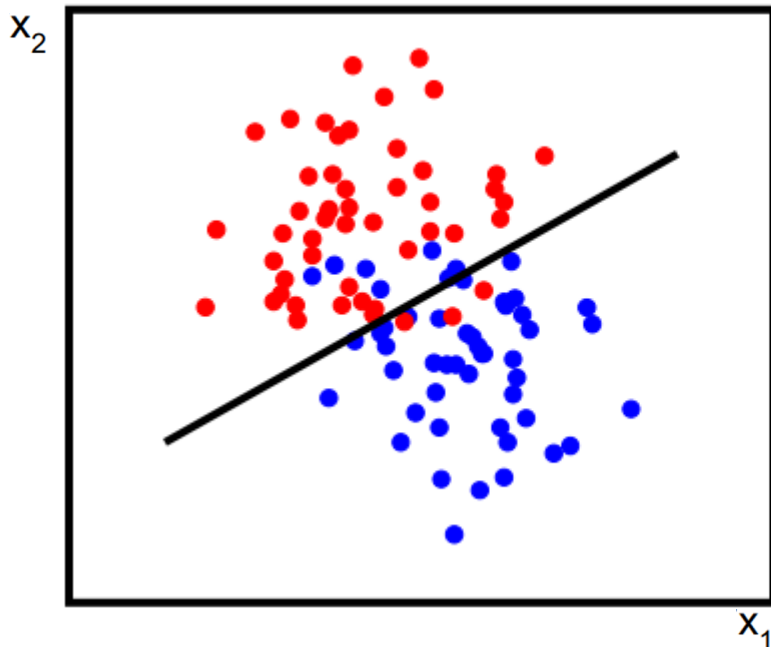


Training the Classifier

- Once we have a smooth measure of quality, we can find the “best” settings for the parameters of

$$r(x_1, x_2) = a \cdot x_1 + b \cdot x_2 + c \quad J(\underline{\theta}) = \frac{1}{m} \sum_j \left(\sigma(r(x^{(j)})) - y^{(j)} \right)^2$$

- Example: 2D feature space $\Leftrightarrow$ parameter space



Finding the Best MSE

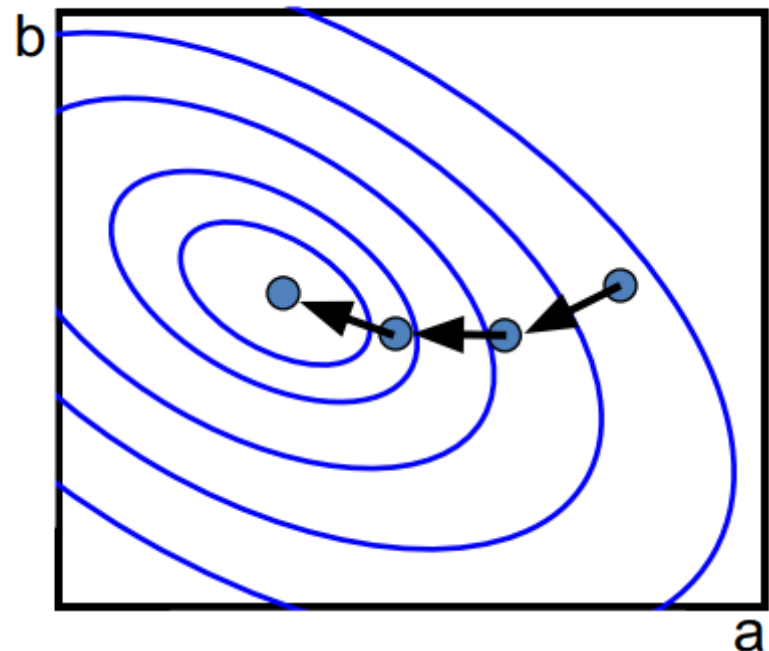
- Once we have a smooth measure of quality, we can find the “best” settings for the parameters of

$$r(x_1, x_2) = a \cdot x_1 + b \cdot x_2 + c \quad J(\underline{\theta}) = \frac{1}{m} \sum_j \left(\sigma(r(x^{(j)})) - y^{(j)} \right)^2$$

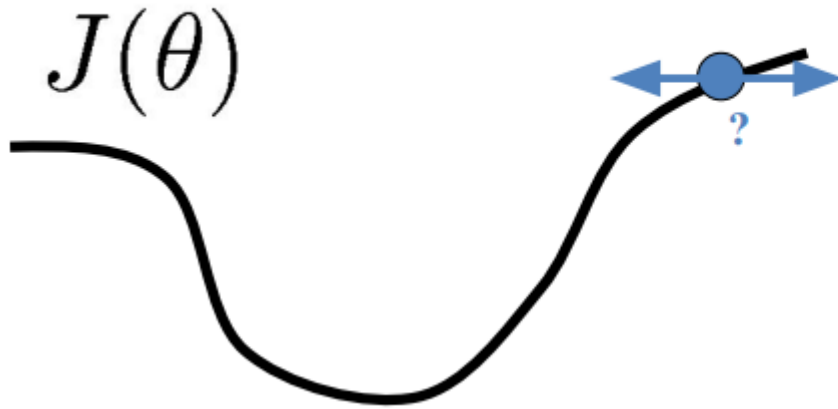
Methods:

- Gradient descent
 - Improve loss by small changes in parameters (“small” = learning rate)

Gradient Descent



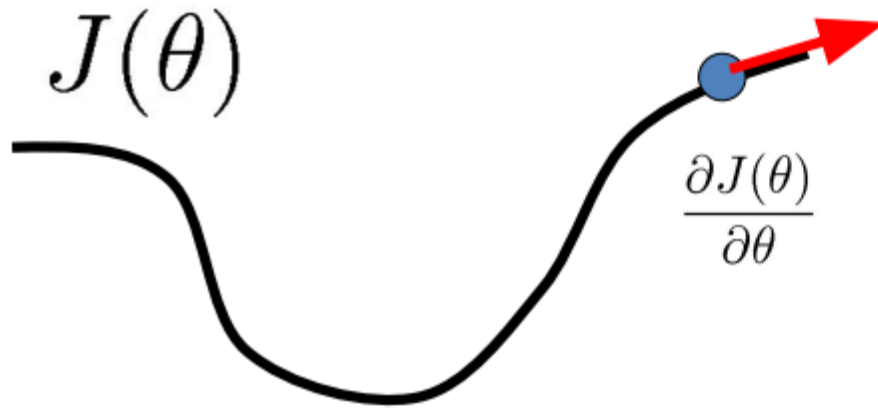
Gradient Descent



- How to change θ to improve $J(\theta)$?
- Choose a direction in which $J(\theta)$ is decreasing

Gradient Descent

$$\frac{\partial J(\theta)}{\partial \theta} = \lim_{h \rightarrow 0} \frac{J(\theta + h) - J(\theta)}{h}$$



- How to change θ to improve $J(\theta)$?
- Choose a direction in which $J(\theta)$ is decreasing
- Derivative $\frac{\partial J(\theta)}{\partial \theta}$
- Positive $\Rightarrow$ increasing
- Negative $\Rightarrow$ decreasing

Gradient Descent

$$\frac{\partial J(\theta)}{\partial \theta} = \lim_{h \rightarrow 0} \frac{J(\theta + h) - J(\theta)}{h}$$

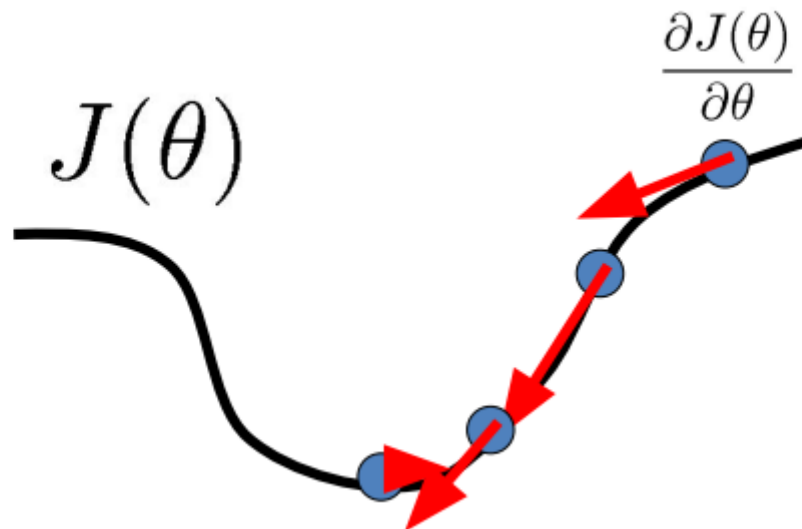
- Initialization
- Step size α
 - Can change over iterations
- Gradient direction
- Stopping condition

Initialize θ

Do{

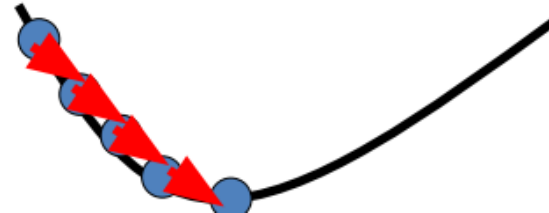
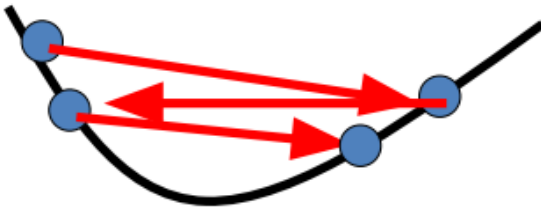
$\theta \leftarrow \theta - \alpha \nabla_{\theta} J(\theta)$

} while $(\alpha \|\nabla_{\theta} J\| > \varepsilon)$



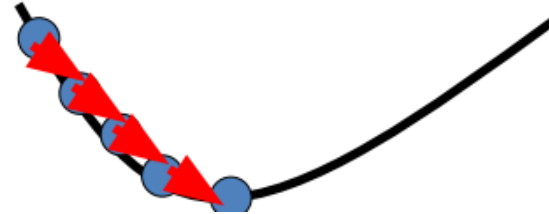
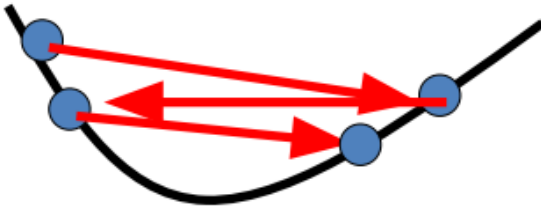
Comments on Gradient Descent

- Very general algorithm
- Step size
 - Too large? Too small? Automatic ways to choose?
 - May want step size to decrease with iteration

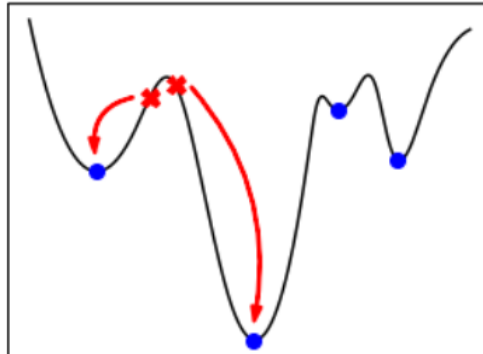


Comments on Gradient Descent

- Very general algorithm
- Step size
 - Too large? Too small? Automatic ways to choose?
 - May want step size to decrease with iteration



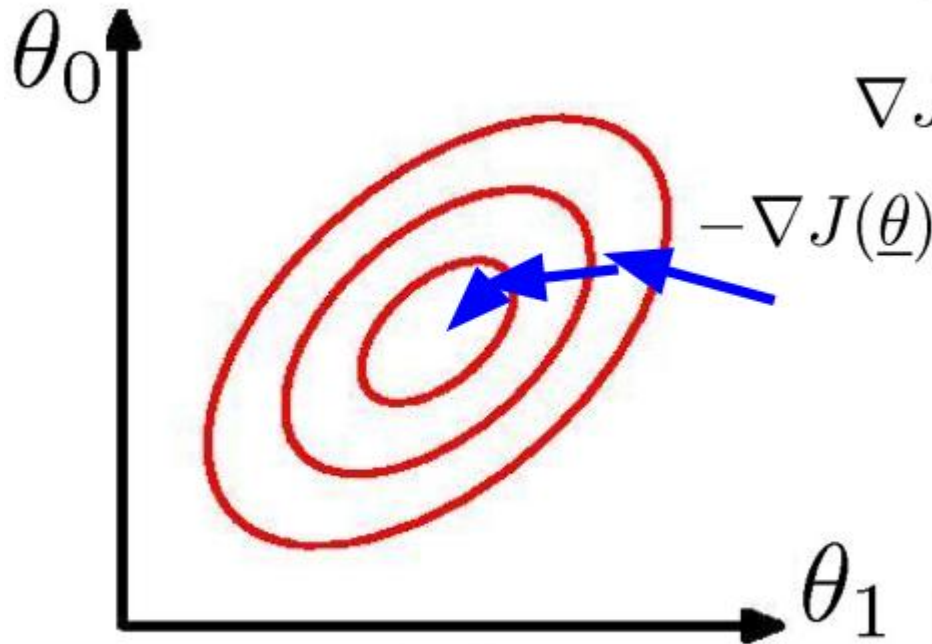
- Local minima
 - Sensitive to starting point



Gradient Descent in More Dimensions

- Gradient vector

$$\nabla J(\underline{\theta}) = \left[\frac{\partial J(\underline{\theta})}{\partial \theta_0} \quad \frac{\partial J(\underline{\theta})}{\partial \theta_1} \quad \dots \right]$$



Indicates direction of steepest ascent
(negative = steepest descent)

Gradient Equations

- MSE (note, depends on function $\sigma(\cdot)$)

$$J(\theta = [a, b, c]) = \frac{1}{m} \sum_i (\sigma(ax_1^{(i)} + bx_2^{(i)} + c) - y^{(i)})^2$$

- What's the derivative with respect to one of the parameters?
 - Recall the chain rule of calculus:

$$\frac{\partial}{\partial a} f(g(h(a))) = f'(g(h(a))) g'(h(a)) h'(a)$$

$$f(g) = (g)^2 \quad \Rightarrow \quad f'(g) = 2(g)$$

$$g(h) = \sigma(h) - y \quad \Rightarrow \quad g'(h) = \sigma'(h)$$

$$h(a) = ax_1^{(i)} + bx_2^{(i)} + c \quad \Rightarrow \quad h'(a) = x_1^{(i)}$$

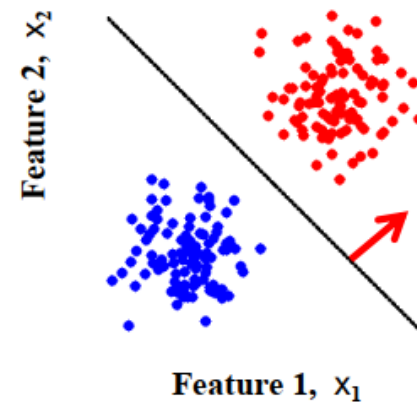
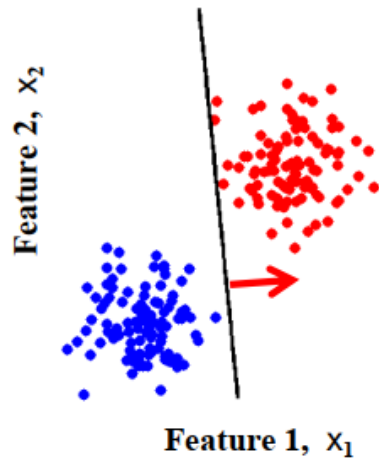
w.r.t. b, c : similar;
replace x_1
with x_2 or 1

$$\frac{\partial J}{\partial a} = \frac{1}{m} \sum_i 2(\sigma(\theta \cdot x^{(i)}) - y^{(i)}) \frac{\partial \sigma(\theta \cdot x^{(i)})}{\partial a} x_1^{(i)}$$

Error between class and prediction Sensitivity of prediction to changes in parameter "a"

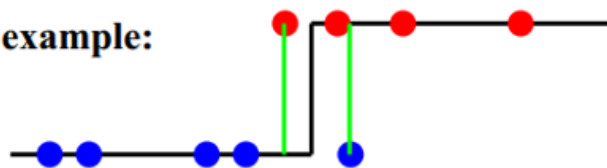
Beyond Misclassification Rate

- Which decision boundary is “better”?
 - Both have zero training error (perfect training accuracy)
 - But, one of them seems intuitively better...

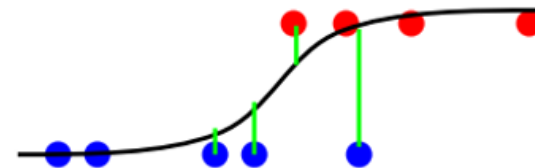


- Side benefit of many “smoothed” error functions
 - Encourages data to be far from the decision boundary
 - See more examples of this principle later...

1D example:



Classification error = $2/9$

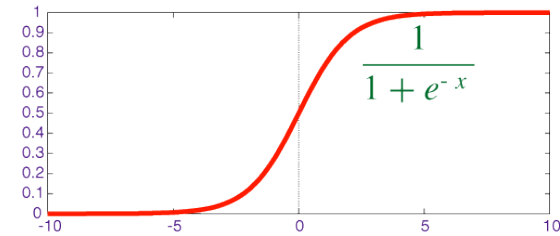


MSE = $(0^2 + 1^2 + .2^2 + .25^2 + .05^2 + \dots)/9$

Class Posterior Probabilities

$$\sigma(\theta^T x) = p(y=1|x) = \text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

- Useful to also know class *probabilities*
- Some notation
 - $p(y=0)$, $p(y=1)$ – class *prior* probabilities
 - How likely is each class in general?
 - $p(x | y=c)$ – class conditional probabilities
 - How likely are observations “x” in that class?
 - $p(y=c | x)$ – class posterior probability
 - How likely is class c *given* an observation x?
- We can compute posterior using Bayes’ rule
 - $p(y=c | x) = p(x|y=c) p(y=c) / p(x)$
- Compute $p(x)$ using sum rule / law of total prob.
 - $p(x) = p(x|y=0) p(y=0) + p(x|y=1)p(y=1)$

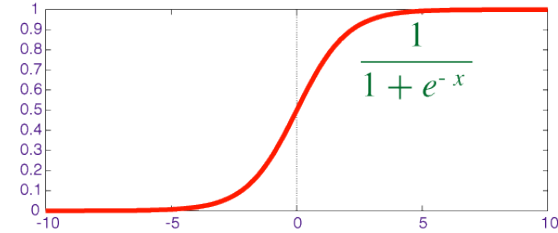


Class Posterior Probabilities

$$\sigma(\underline{\theta}^T \mathbf{x}) = p(y=1|x) = \text{sigmoid}(\mathbf{x}) = \frac{1}{1 + e^{-x}}$$

- Consider comparing two classes
 - $p(\mathbf{x} | y=0) * p(y=0)$ vs $p(\mathbf{x} | y=1) * p(y=1)$

$$p(y=c | \mathbf{x}) = p(\mathbf{x}|y=c) p(y=c) / p(\mathbf{x})$$

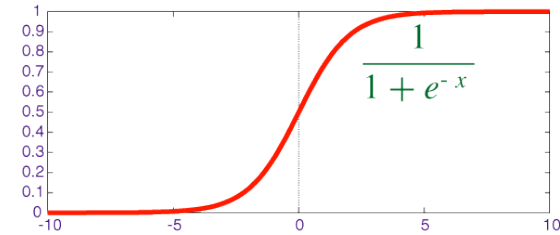


Class Posterior Probabilities

$$\sigma(\underline{\theta}^T x) = p(y=1|x) = \text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

- Consider comparing two classes

- $p(x | y=0) * p(y=0)$ vs $p(x | y=1) * p(y=1)$
- Write probability of each class as
- $p(y=0 | x) = p(y=0, x) / p(x)$
- $= p(y=0, x) / (p(y=0,x) + p(y=1,x))$
- $= 1 / (1 + \exp(-a))$ (**)

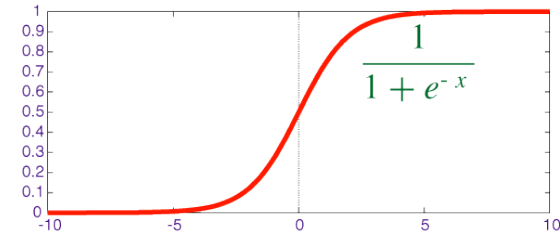


Class Posterior Probabilities

$$\sigma(\theta^T x) = p(y=1|x) = \text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

- Consider comparing two classes

- $p(x | y=0) * p(y=0)$ vs $p(x | y=1) * p(y=1)$
- Write probability of each class as
- $p(y=0 | x) = p(y=0, x) / p(x)$
- $= p(y=0, x) / (p(y=0,x) + p(y=1,x))$
- $= 1 / (1 + \exp(-a))$ (**)



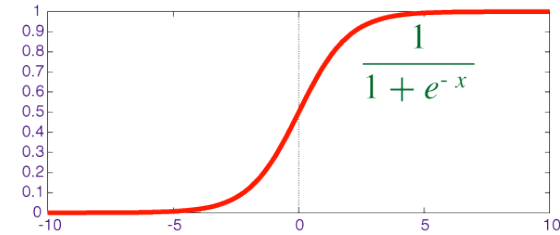
$$\begin{aligned} p(y=0|x) &= p(y=0,x) / p(x) \\ &= p(y=0,x) / (p(y=0,x)+p(y=1,x)) \\ &= 1 / (1 + p(y=1,x)/p(y=0,x)) \\ &= 1 / (1 + p(x|y=1)p(y=1)/p(x|y=0)p(y=0)) \\ &= 1 / (1 + \exp(\log(p(x|y=1)p(y=1)/p(x|y=0)p(y=0)))) \\ &= 1 / (1 + \exp(- \log(p(x|y=0)p(y=0)/p(x|y=1)p(y=1)))) \end{aligned}$$

Class Posterior Probabilities

$$\sigma(\theta^T x) = p(y=1|x) = \text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

- Consider comparing two classes

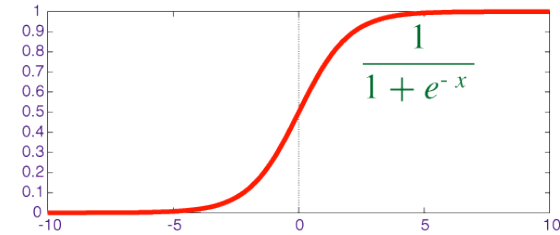
- $p(x | y=0) * p(y=0)$ vs $p(x | y=1) * p(y=1)$
- Write probability of each class as
- $p(y=0 | x) = p(y=0, x) / p(x)$
- $= p(y=0, x) / (p(y=0,x) + p(y=1,x))$
- $= 1 / (1 + \exp(-a))$ (**)
- $a = \log [p(x|y=0) p(y=0) / p(x|y=1) p(y=1)]$
- (**) called the logistic function, or logistic sigmoid.



$$\begin{aligned} p(y=0|x) &= p(y=0,x) / p(x) \\ &= p(y=0,x) / (p(y=0,x)+p(y=1,x)) \\ &= 1 / (1 + p(y=1,x)/p(y=0,x)) \\ &= 1 / (1 + p(x|y=1)p(y=1)/p(x|y=0)p(y=0)) \\ &= 1 / (1 + \exp(\log(p(x|y=1)p(y=1)/p(x|y=0)p(y=0)))) \\ &= 1 / (1 + \exp(- \log(p(x|y=0)p(y=0)/p(x|y=1)p(y=1)))) \end{aligned}$$

Logistic Regression

$$\sigma(\underline{\theta} x^T) = p(y=1|x) = \text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$



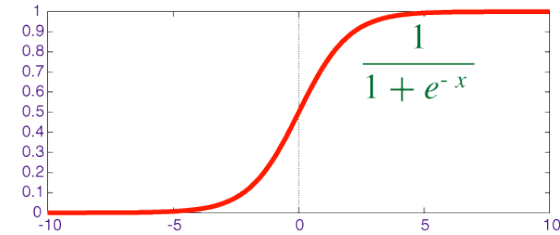
- Interpret $\sigma(\underline{\theta} x^T)$ as a probability that $y = 1$
- Use a negative log-likelihood loss function
 - If $y = 1$, cost is $-\log \Pr[y=1] = -\log \sigma(\underline{\theta} x^T)$
 - If $y = 0$, cost is $-\log \Pr[y=0] = -\log (1 - \sigma(\underline{\theta} x^T))$

- Can write this succinctly: i.e., the binary cross-entropy (BCE) loss

$$J(\underline{\theta}) = -\frac{1}{m} \left(\underbrace{\sum_i y^{(i)} \log \sigma(\underline{\theta} \cdot x^{(i)})}_{\text{Nonzero only if } y=1} + \underbrace{(1 - y^{(i)}) \log (1 - \sigma(\underline{\theta} \cdot x^{(i)}))}_{\text{Nonzero only if } y=0} \right)$$

Logistic Regression

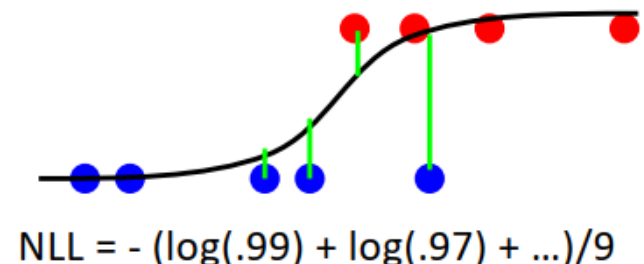
$$\sigma(\underline{\theta} x^T) = p(y=1|x) = \text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$



- Interpret $\sigma(\underline{\theta} x^T)$ as a probability that $y = 1$
- Use a negative log-likelihood loss function
 - If $y = 1$, cost is $-\log \Pr[y=1] = -\log \sigma(\underline{\theta} x^T)$
 - If $y = 0$, cost is $-\log \Pr[y=0] = -\log (1 - \sigma(\underline{\theta} x^T))$

- Can write this succinctly: i.e., the binary cross-entropy (BCE) loss

$$J(\underline{\theta}) = -\frac{1}{m} \left(\underbrace{\sum_i y^{(i)} \log \sigma(\underline{\theta} \cdot x^{(i)})}_{\text{Nonzero only if } y=1} + \underbrace{(1 - y^{(i)}) \log (1 - \sigma(\underline{\theta} \cdot x^{(i)}))}_{\text{Nonzero only if } y=0} \right)$$



Gradient Equations

- Logistic neg-log likelihood loss:

i.e., the binary cross-entropy (BCE) loss

$$J(\underline{\theta}) = -\frac{1}{m} \left(\sum_i y^{(i)} \log \sigma(\theta \cdot x^{(i)}) + (1 - y^{(i)}) \log(1 - \sigma(\theta \cdot x^{(i)})) \right)$$

- What's the derivative with respect to one of the parameters?

$$\theta = [\dots, a, \dots]$$

$$\begin{aligned} \frac{\partial J}{\partial a} &= -\frac{1}{m} \left(\sum_i y^{(i)} \frac{1}{\sigma(\theta \cdot x^{(i)})} \frac{\partial \sigma(\theta \cdot x^{(i)})}{\partial a} x_1^{(i)} + (1 - y^{(i)}) \dots \right) \\ &= -\frac{1}{m} \left(\sum_i y^{(i)} (1 - \sigma(\theta \cdot x^{(i)})) x_1^{(i)} + (1 - y^{(i)}) \dots \right) \end{aligned}$$

Summary

- Linear classifier $\Leftrightarrow$ perceptron
- Measuring quality of a decision boundary
 - Error rate (0/1 loss)
 - Logistic sigmoid + MSE criterion
 - Logistic Regression
- Learning the weights of a linear classifier from data
 - Reduces to an optimization problem
 - Perceptron algorithm
 - For MSE or Logistic NLL, we can do gradient descent
 - Gradient equations & update rules

Outline

- Linear Classification and Perceptron Learning
- Logistic Regression and Gradient-Based Learning
- **Multi-Class Classification**

Multi-Class Linear Models

- What about multiple classes? One option:

- Define one linear response per class
- Choose class with the largest response

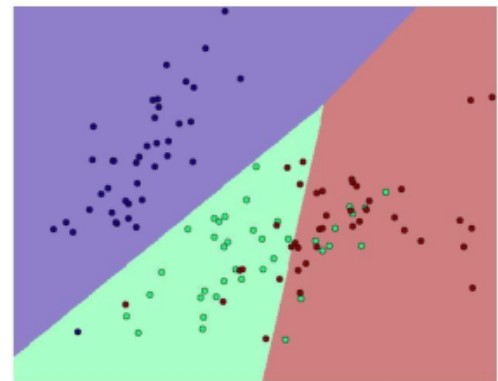
$$\theta = \begin{bmatrix} \theta_{00} & \dots & \theta_{0n} \\ \vdots & \ddots & \vdots \\ \theta_{C0} & \dots & \theta_{Cn} \end{bmatrix}$$

$$f(x; \theta) = \arg \max_c \theta_c \cdot x^T$$

- Boundary between two classes, c vs. c' ?

$$= \begin{cases} c & \text{if } \theta_c \cdot x^T > \theta_{c'} x^T \\ c' & \text{otherwise} \end{cases} \Leftrightarrow (\theta_c - \theta_{c'}) x^T > 0$$

- Linear boundary: $(\theta_c - \theta_{c'}) x^T = 0$



Multi-Class Linear Models

- Define the probability of each class:

$$p(Y = y|X = x) = \frac{\exp(\theta_y \cdot x^T)}{\sum_c \exp(\theta_c \cdot x^T)}$$

(Y binary = logistic regression)

- Then, the NLL loss function is:

$$J(\theta) = -\frac{1}{m} \sum_i \log p(y^{(i)}|x^{(i)})$$

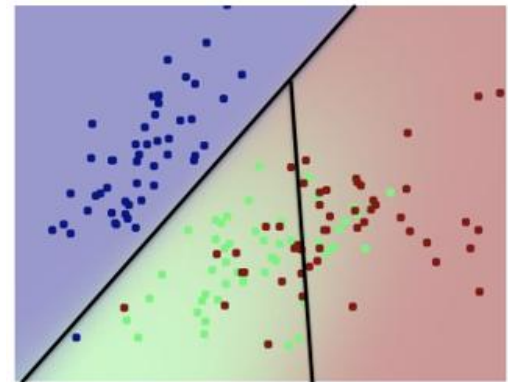
- P: “confidence” of each class
 - Soft decision value
- Decision: predict most probable
 - Linear decision boundary
- Convex loss function

Recall binary CE loss

$$J(\theta) = -\frac{1}{m} \left(\underbrace{\sum_i y^{(i)} \log \sigma(\theta \cdot x^{(i)})}_{\text{Nonzero only if } y=1} + \underbrace{\sum_i (1-y^{(i)}) \log (1-\sigma(\theta \cdot x^{(i)}))}_{\text{Nonzero only if } y=0} \right)$$

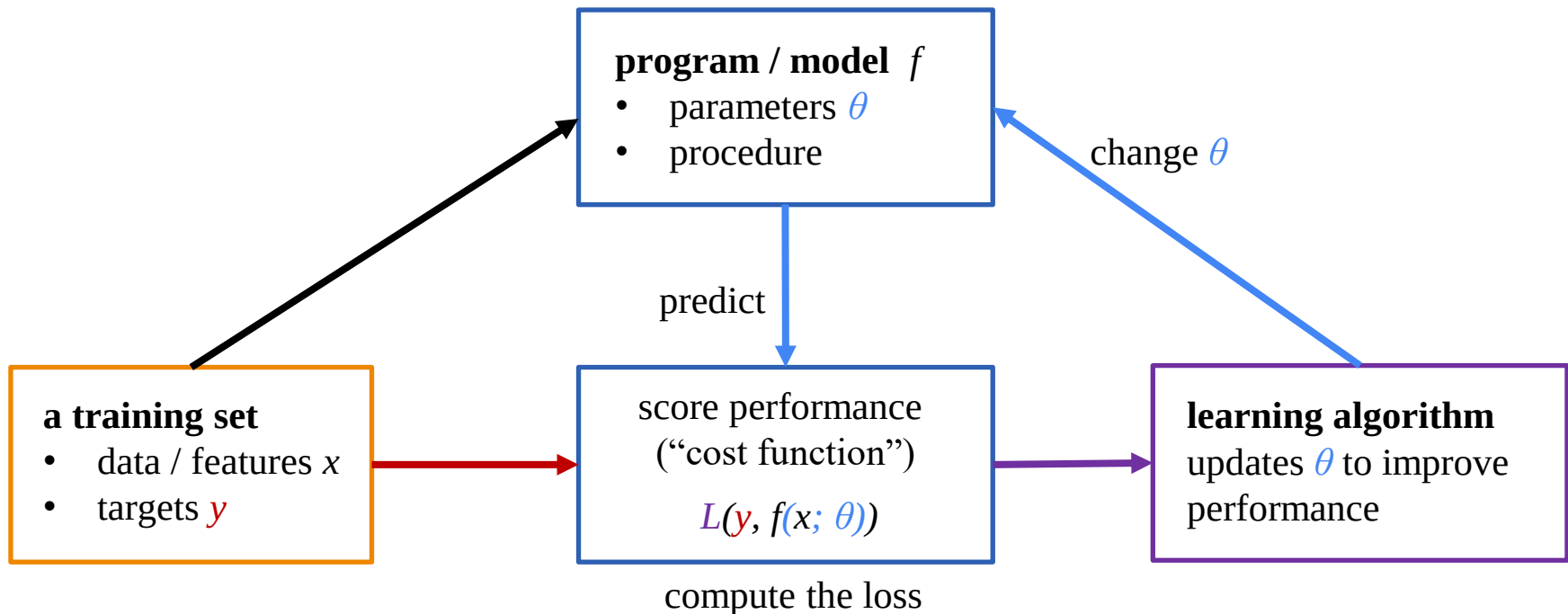
Nonzero only if y=1

Nonzero only if y=0



Review

- Linear Classification and Perceptron Learning
- Logistic Regression and Gradient-Based Learning
- Multi-Class Classification

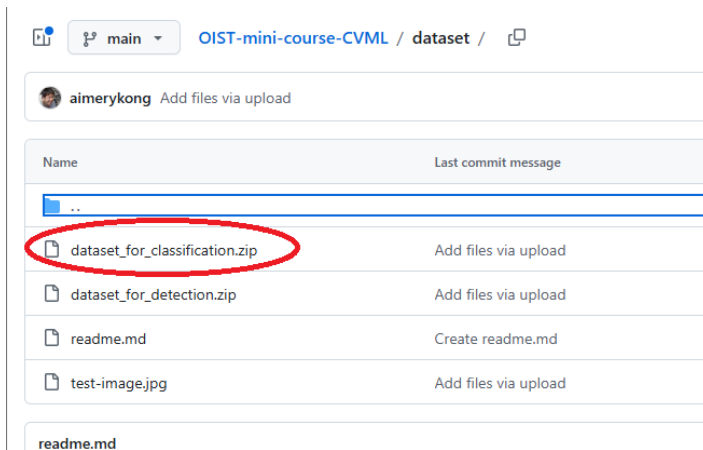


Hands-on exercise

<https://github.com/aimerykong/OIST-mini-course-CVML>

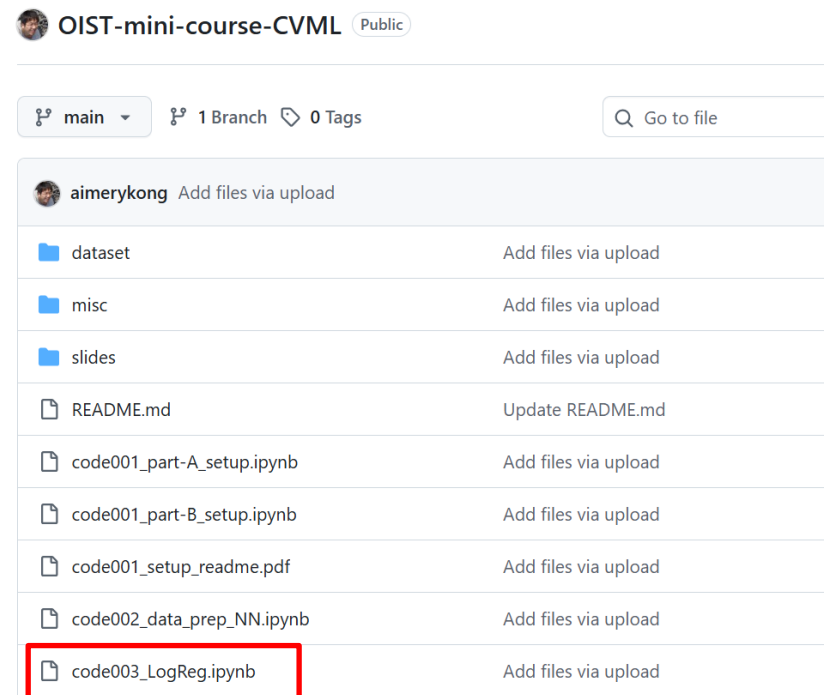
Today's coding exercise:

- Understand the importance of using better features
 - Understand the importance of using better models
 - Use third-party packages to train linear classifiers
-
- Download the zip file (the same one as last week!) and upload to your gdrive
 - Download and upload the jupyter notebook “code003_LogReg.ipynb” to colab



The screenshot shows the GitHub interface for the repository 'OIST-mini-course-CVML' at the 'dataset' directory. The file 'dataset_for_classification.zip' is highlighted with a red circle. The table below lists the files in the directory:

Name	Last commit message
..	
dataset_for_classification.zip	Add files via upload
dataset_for_detection.zip	Add files via upload
readme.md	Create readme.md
test-image.jpg	Add files via upload



The screenshot shows the GitHub interface for the repository 'OIST-mini-course-CVML' at the root directory. The file 'code003_LogReg.ipynb' is highlighted with a red box. The table below lists the files in the directory:

Name	Last commit message
dataset	Add files via upload
misc	Add files via upload
slides	Add files via upload
README.md	Update README.md
code001_part-A_setup.ipynb	Add files via upload
code001_part-B_setup.ipynb	Add files via upload
code001_setup_readme.pdf	Add files via upload
code002_data_prep_NN.ipynb	Add files via upload
code003_LogReg.ipynb	Add files via upload

